



OBJECT-ORIENTED PROGRAMMING (OOP) FOR IoT

Course: IoT Short Course

Topic: Writing Scalable, Reusable Code for Multiple Sensors/Devices

1. INTRODUCTION TO OOP

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **"objects"** rather than functions and logic. An object is a data structure containing **attributes** (data) and **methods** (functions that operate on that data).

Why OOP in IoT?

In IoT, we handle dozens of different sensors, actuators, and communication protocols. OOP helps us:

- **Reuse code** (write once, use for all sensors).
 - **Scale systems** (add 10 new sensors without rewriting code).
 - **Hide complexity** (the user of a sensor doesn't need to know the internal wiring).
 - **Maintain code** (fix bugs in one place, updates apply everywhere).
-

2. REAL-WORLD APPLICATIONS OF OOP IN IoT

A) Classes & Objects

Real-World Example: Smart Home Platforms (Google Home, Amazon Alexa, Apple HomeKit).

- **Class:** `SmartDevice`
- **Objects:** Your specific "Living Room Light", "Kitchen Fan", and "Bedroom Thermostat".
- **Why:** The platform treats every device as an object that has an `On/Off` state, a `Location`, and a `Name`. This allows the app to display a list of devices dynamically without knowing what each one is specifically.

B) Inheritance

Real-World Example: Fitbit / Apple Watch Health Monitoring.

- **Parent Class:** `Sensor` (has properties like `read_data()` , `unit`).
- **Child Classes:** `HeartRateSensor` (inherits basic sensor functions but adds `calculate_bpm()`), `AccelerometerSensor` (inherits basic functions but adds `count_steps()`), `SpO2Sensor` (adds `check_oxygen_level()`).
- **Why:** The underlying hardware communication code is written once in the parent `Sensor` class. The specific health features are added in the children.

C) Polymorphism (Method Overriding)

Real-World Example: Smart Factory (IIoT - Industrial Internet of Things).

- **Scenario:** A central control system wants to "Start" all machines for the morning shift.
- **Parent Class:** `Machine` has a method `start_sequence()` .
- **Child Classes:** `ConveyorBelt` starts slowly and speeds up; `RoboticArm` runs a self-diagnostic first; `CNC_Machine` pre-heats before moving.
- **Why:** The central system just sends a `start_sequence()` command to every object. Each machine knows *how* to start itself, so the central system doesn't need to know the specifics of 50 different types of machinery.

D) Encapsulation (Private Attributes)

Real-World Example: Medical IoT (Smart Syringe Pumps / Patient Monitors).

- **Scenario:** The dosage or medical reading must not be accidentally changed by a nurse touching the wrong screen.
- **Private Attribute:** `__medication_dosage` or `__calibration_factor` .
- **Why:** By making critical calibration data private and only accessible via a specific method (e.g., `update_dosage(verify_password)`), the device ensures that only authorized logic changes the medical data, preventing life-threatening errors.

E) @Property Decorators (Data Validation)

Real-World Example: Smart Agriculture (Greenhouse Automation).

- **Scenario:** A soil pH sensor must send data between 0 and 14. If the sensor malfunctions and sends "99", it shouldn't activate the acid/alkali dosing pumps.
- **Property Decorator:** The `value` property checks the reading before storing it.

- **Why:** If `value > 14`, the setter raises an `ValueError` and logs an alert instead of poisoning the crops. It ensures the system only acts on valid data.
-

3. CORE OOP CONCEPTS EXPLAINED

3.1 Classes and Objects

- **Class:** A blueprint (e.g., a template for a "Car").
- **Object:** An instance of that class (e.g., the actual "Red Tesla" parked in your garage).

3.2 The `__init__` Method (Constructor)

The `__init__` method is the **constructor**. It is automatically called when you create a new object. It sets up the initial state of the object.

3.3 Inheritance

Inheritance allows a child class to **acquire** all the attributes and methods of a parent class.

- **Parent:** `Sensor`
- **Child:** `TempSensor`, `HumiditySensor`
- `super().__init__()` is used to call the parent's constructor so you don't have to rewrite common code.

3.4 Encapsulation (Public, Protected, Private)

- **Public (`self.name`):** Accessible from anywhere.
- **Protected (`self._name`):** Indicates it should not be accessed directly (convention, not strictly enforced).
- **Private (`self.__name`):** **Name Mangling** occurs. Python renames it to `_Class__name` making it hard to accidentally access.

3.5 Polymorphism

The ability of different classes to respond to the same method call in their own way. (e.g., Both `TempSensor` and `HumiditySensor` have a `read()` method, but they do different things internally).

3.6 Property Decorators (@property)

The @property decorator allows you to define methods that can be accessed like attributes. This allows you to **validate** data before it is assigned.

4. COMPLETE CODE IMPLEMENTATION

This is the production-ready code for Week 2. It demonstrates all the concepts listed above.

Step 1: The Base SmartDevice Class

```
# =====
# BASE CLASS: SmartDevice
# =====

import random
import time
import json
from datetime import datetime

class SmartDevice:
    """Base class for all IoT devices"""

    # Class attribute (shared by all instances)
    device_count = 0

    def __init__(self, device_id, name, location, manufacturer="Generic")
        """
        Initialize a SmartDevice

        Args:
            device_id (str): Unique identifier
            name (str): Human-readable name
            location (str): Physical location (e.g., "Living Room")
            manufacturer (str): Device manufacturer
        """
        # Public attributes
        self.device_id = device_id
        self.name = name
        self.location = location
```

```

self.manufacturer = manufacturer

# Private attributes (Encapsulation)
self.__internal_firmware_version = "v1.0.0"
self.is_on = False
self.readings = []
self.max_readings = 100
self.created_at = datetime.now()
self.last_read_at = None

# Update class counter
SmartDevice.device_count += 1

print(f"🔧 Created {self.__class__.__name__}: {name} (ID: {device_id})")

# -----
# PUBLIC METHODS
# -----

def turn_on(self):
    """Turn the device ON"""
    self.is_on = True
    print(f"🟢 {self.name} turned ON")
    return True

def turn_off(self):
    """Turn the device OFF"""
    self.is_on = False
    print(f"🔴 {self.name} turned OFF")
    return True

def add_reading(self, value):
    """Add a sensor reading with timestamp"""
    reading = {
        "timestamp": datetime.now().isoformat(),
        "value": value
    }
    self.readings.append(reading)
    self.last_read_at = datetime.now()

    if len(self.readings) > self.max_readings:
        self.readings = self.readings[-self.max_readings:]

    return reading

```

```

def get_latest_reading(self):
    """Return the most recent reading"""
    if not self.readings:
        return None
    return self.readings[-1]

def get_average_reading(self):
    """Calculate average of readings"""
    if not self.readings:
        return 0
    values = [r["value"] for r in self.readings]
    return sum(values) / len(values)

def to_dict(self):
    """Convert device to dictionary (for JSON saving)"""
    return {
        "device_id": self.device_id,
        "name": self.name,
        "location": self.location,
        "type": self.__class__.__name__,
        "is_on": self.is_on,
        "readings": self.readings[-5:],
        "created_at": self.created_at.isoformat()
    }

def __str__(self):
    return f"{self.__class__.__name__}: {self.name} at {self.location}"

```

Step 2: The Base **Sensor** Class (Inheritance)

```

# =====
# PARENT CLASS: BaseSensor (Inherits from SmartDevice)
# =====

class BaseSensor(SmartDevice):
    """Base class for all sensors"""

    def __init__(self, device_id, name, location, sensor_type, unit):
        # Call parent constructor
        super().__init__(device_id, name, location)

```

```

    # Sensor-specific attributes
    self.sensor_type = sensor_type
    self.unit = unit
    self.last_reading = None
    self._calibration_offset = 0.0 # Protected

# =====
# PROPERTY DECORATOR (Validation)
# =====

@property
def calibration_offset(self):
    """Get the calibration offset"""
    return self._calibration_offset

@calibration_offset.setter
def calibration_offset(self, value):
    """
    Set calibration offset with validation
    Ensures offset is within reasonable range (-5 to +5)
    """
    if not isinstance(value, (int, float)):
        raise TypeError("Offset must be a number!")

    if value < -5 or value > 5:
        raise ValueError("Offset must be between -5 and 5!")

    self._calibration_offset = value
    print(f"🔧 {self.name} calibrated with offset: {value}")

# =====
# VIRTUAL METHOD (To be overridden)
# =====

def read_sensor(self):
    """Virtual method - child classes MUST implement this"""
    raise NotImplementedError("Child class must implement read_sensor")

def get_reading(self):
    """Get reading from sensor and store it"""
    value = self.read_sensor()
    self.last_reading = value
    self.add_reading(value)
    return value

```

```
def __str__(self):
    return f"BaseSensor[{self.device_id}]: {self.sensor_type}"
```

Step 3: Child Class 1 - Temperature Sensor (Concrete Implementation)

```
# =====
# CHILD CLASS 1: TemperatureSensor
# =====

class TemperatureSensor(BaseSensor):
    """Temperature sensor implementation"""

    def __init__(self, device_id, name, location, min_temp=-40, max_temp=
        super().__init__(device_id, name, location, "Temperature", "°C")
        self.min_temp = min_temp
        self.max_temp = max_temp
        self._current_temp = 25.0 # Starting point for simulation

# =====
# PROPERTY DECORATOR (Range Validation)
# =====

@property
def current_temp(self):
    """Get current temperature"""
    return self._current_temp

@current_temp.setter
def current_temp(self, value):
    """Set current temperature with range validation"""
    if value < self.min_temp:
        self._current_temp = self.min_temp
    elif value > self.max_temp:
        self._current_temp = self.max_temp
    else:
        self._current_temp = round(value, 2)

# =====
# POLYMORPHISM: Overriding the parent method
# =====
```



```

def read_sensor(self):
    """Simulate reading real hardware (e.g., DS18B20 or DHT22)"""
    # Simulate natural fluctuation
    variation = random.uniform(-0.5, 0.5)
    raw_value = self._current_temp + variation

    # Apply calibration offset
    corrected_value = raw_value + self._calibration_offset

    # Clamp to physical limits
    self.current_temp = corrected_value # Uses the setter for validation

    return self._current_temp

# =====
# UNIQUE METHOD (Not in parent)
# =====

def to_fahrenheit(self):
    """Convert to Fahrenheit"""
    return round((self._current_temp * 9/5) + 32, 2)

def __str__(self):
    return f"🌡️ TempSensor: {self.name} @ {self.current_temp}°C"

```

Step 4: Child Class 2 - Humidity Sensor

```

# =====
# CHILD CLASS 2: HumiditySensor
# =====

class HumiditySensor(BaseSensor):
    """Humidity sensor implementation"""

    def __init__(self, device_id, name, location):
        super().__init__(device_id, name, location, "Humidity", "%")
        self._current_humidity = 50.0

# =====
# PROPERTY DECORATOR (0-100% Validation)
# =====

```

```

@property
def current_humidity(self):
    return self._current_humidity

@current_humidity.setter
def current_humidity(self, value):
    """Ensure humidity stays between 0% and 100%"""
    if value < 0:
        self._current_humidity = 0.0
    elif value > 100:
        self._current_humidity = 100.0
    else:
        self._current_humidity = round(value, 1)

# =====
# POLYMORPHISM: Overriding the parent method
# =====

def read_sensor(self):
    """Simulate reading humidity sensor"""
    # Simulate fluctuation
    variation = random.uniform(-2.0, 2.0)
    raw_value = self._current_humidity + variation

    # Apply calibration
    corrected_value = raw_value + self._calibration_offset

    # Clamp to 0-100%
    self.current_humidity = corrected_value

    return self._current_humidity

# =====
# UNIQUE METHOD
# =====

def calculate_dew_point(self, temperature):
    """Calculate dew point using temperature and humidity"""
    dew_point = temperature - ((100 - self._current_humidity) / 5)
    return round(dew_point, 2)

def __str__(self):
    return f"💧 HumidSensor: {self.name} @ {self.current_humidity}%"

```

Step 5: Child Class 3 - Motion Sensor (PIR)

```
# =====
# CHILD CLASS 3: MotionSensor (PIR)
# =====

class MotionSensor(BaseSensor):
    """Motion (PIR) sensor implementation"""

    def __init__(self, device_id, name, location):
        super().__init__(device_id, name, location, "Motion", "Binary")
        self.motion_count = 0
        self.last_motion_time = None
        self._is_motion = False

# =====
# POLYMORPHISM: Overriding the parent method
# =====

    def read_sensor(self):
        """Simulate PIR sensor reading (1=Detected, 0=No Motion)"""
        # 15% chance of detecting motion
        detection = random.random() < 0.15

        if detection:
            self._is_motion = True
            self.motion_count += 1
            self.last_motion_time = datetime.now()
        else:
            self._is_motion = False

        # Store as 1 or 0
        return 1 if self._is_motion else 0

# =====
# UNIQUE METHODS
# =====

    def get_status(self):
        """Get human-readable status"""
        if self._is_motion:
            return "🧑 Motion Detected!"
        return "🟢 No Motion"
```

```

def has_recent_motion(self, seconds=30):
    """Check if motion detected in last X seconds"""
    if self.last_motion_time is None:
        return False

    elapsed = (datetime.now() - self.last_motion_time).total_seconds()
    return elapsed < seconds

def __str__(self):
    return f"👤 MotionSensor: {self.name} (Detections: {self.motion_c

```

Step 6: Main Application (Assignment Submission)

This is the final execution script. It instantiates the objects and stores the readings in a dictionary.

```

# =====
# FINAL ASSIGNMENT: Smart Device System
# =====

print("=" * 60)
print("📡 IoT SMART DEVICE SYSTEM")
print("=" * 60)

# =====
# STEP 1: INSTANTIATE DEVICES
# =====

# Temperature Sensor
temp_sensor = TemperatureSensor("SENS-001", "Living Room Temp", "Living R
# Set calibration offset using the @property setter (Validation)
temp_sensor.calibration_offset = 0.5

# Humidity Sensor
humid_sensor = HumiditySensor("SENS-002", "Basement Humidity", "Basement"
# This will trigger validation (value should be between -5 and 5)
try:
    humid_sensor.calibration_offset = 10.0 # INVALID - will raise error
except ValueError as e:
    print(f"✅ Validation caught error: {e}")

# Motion Sensor

```

```

motion_sensor = MotionSensor("SENS-003", "Hallway Motion", "Hallway")

print("\n" + "=" * 60)
print("✅ DEVICES CREATED SUCCESSFULLY")
print("=" * 60)

# =====
# STEP 2: SIMULATE DATA COLLECTION
# =====

print("\n📊 SIMULATING DATA COLLECTION...")

all_readings = {
    "devices": [],
    "device_readings": {}
}

# Read sensors 10 times
for i in range(10):
    print(f"\n--- Reading {i+1} ---")

    # Read Temperature
    temp_val = temp_sensor.get_reading()
    print(f"🌡️ Temp: {temp_val}°C")

    # Read Humidity
    humid_val = humid_sensor.get_reading()
    print(f"💧 Humidity: {humid_val}%")

    # Read Motion
    motion_val = motion_sensor.get_reading()
    status = "Motion" if motion_val == 1 else "No Motion"
    print(f"🚶 Motion: {status}")

    # Store in dictionary (ASSIGNMENT REQUIREMENT)
    all_readings["device_readings"][f"Reading_{i+1}"] = {
        "timestamp": datetime.now().isoformat(),
        "temperature": temp_val,
        "humidity": humid_val,
        "motion_detected": motion_val
    }

    time.sleep(0.5)

```

```

# =====
# STEP 3: STORE DEVICE OBJECTS IN DICTIONARY
# =====

# Store the device objects in a dictionary
device_catalogue = {
    "living_room_temp": temp_sensor,
    "basement_humidity": humid_sensor,
    "hallway_motion": motion_sensor
}

all_readings["devices"] = list(device_catalogue.keys())

print("\n" + "=" * 60)
print("📖 DEVICE CATALOGUE (Objects stored in Dictionary)")
print("=" * 60)

for name, device in device_catalogue.items():
    print(f"♦ {name}: {device}")

# =====
# STEP 4: DISPLAY FINAL DATA
# =====

print("\n" + "=" * 60)
print("📄 FINAL DATA SUMMARY")
print("=" * 60)

# 1. Display latest readings
print("\n📊 Latest Readings:")
for name, device in device_catalogue.items():
    latest = device.get_latest_reading()
    if latest:
        print(f"    {name}: {latest}")

# 2. Display averages
print("\n📈 Averages:")
print(f"Avg Temperature: {temp_sensor.get_average_reading():.2f}°C")
print(f"Avg Humidity: {humid_sensor.get_average_reading():.1f}%")
print(f>Total Motion Detections: {motion_sensor.motion_count}")

# 3. Display the full dictionary (Assignment requirement)
print("\n📖 Full Data Dictionary:")
print(json.dumps(all_readings, indent=2, default=str))

```

```

# =====
# STEP 5: DEMONSTRATE ENCAPSULATION
# =====

print("\n" + "=" * 60)
print("🔒 ENCAPSULATION DEMONSTRATION")
print("=" * 60)

print("\n♦ Trying to access private attribute directly:")
try:
    print(temp_sensor.__internal_firmware_version)  # ❌ Fails
except AttributeError as e:
    print(f"❌ Error: {e}")

print("\n♦ Private attribute name is mangled:")
print(f"    Actually stored as: {temp_sensor._SmartDevice__internal_firmware_version}")

print("\n✅ Using validation via @property:")
print(f"    Current calibration: {temp_sensor.calibration_offset}")
temp_sensor.calibration_offset = -1.2  # Works (within -5 to 5)
print(f"    Updated calibration: {temp_sensor.calibration_offset}")

# =====
# STEP 6: DEMONSTRATE POLYMORPHISM
# =====

print("\n" + "=" * 60)
print("🔄 POLYMORPHISM DEMONSTRATION")
print("=" * 60)

print("\n♦ All devices have the same method 'turn_on()':")
for name, device in device_catalogue.items():
    print(f"    Turning on {name}:")
    device.turn_on()  # Same method name, different behavior possible

print("\n♦ All devices have the same method 'to_dict()' but different outputs:")
for name, device in device_catalogue.items():
    data = device.to_dict()
    print(f"    {name}: {list(data.keys())}")

# =====
# STEP 7: SAVE TO FILE (Bonus)
# =====

```

```
print("\n" + "=" * 60)
print("💾 SAVING DATA TO JSON")
print("=" * 60)

try:
    with open("assignment_data.json", "w") as f:
        json.dump(all_readings, f, indent=2, default=str)
        print("✅ Data saved to assignment_data.json")
except Exception as e:
    print(f"❌ Could not save: {e}")

print("\n" + "=" * 60)
print("✅ ASSIGNMENT COMPLETED SUCCESSFULLY!")
print("=" * 60)
```

5. TROUBLESHOOTING COMMON ERRORS

Error Type	Example	Solution
AttributeError	<code>obj.__private_var</code>	Access via getter method or mangled name <code>_Class__private_var</code>
TypeError	Setting string where int expected	Use <code>@property.setter</code> to validate types
ValueError	Setting value out of range	Use validation in setter to clamp or raise error
NotImplementedError	<code>read_sensor()</code> called on base class	Ensure you instantiate child classes, not the parent

6. CHEAT SHEET (Quick Reference)

Concept	Syntax	Use Case
Class	<code>class SmartDevice:</code>	Blueprint
Object	<code>obj = SmartDevice()</code>	Actual device
Constructor	<code>def __init__(self):</code>	Initialize
Inheritance	<code>class TempSensor(SmartDevice):</code>	Reuse parent
Super()	<code>super().__init__()</code>	Call parent

Concept	Syntax	Use Case
Protected	<code>self._variable</code>	Internal use
Private	<code>self.__variable</code>	Name mangling
Property	<code>@property @var.setter</code>	Validation
Polymorphism	Override parent methods	Custom behavior

July 17

Submission Checklist:

- ☐ `SmartDevice` parent class created
- ☐ 3 child classes created
- ☐ Objects instantiated
- ☐ Objects stored in a dictionary
- ☐ Readings stored in a dictionary
- ☐ Code runs without errors
- ☐ Encapsulation demonstrated
- ☐ Polymorphism demonstrated